Project Report: SpendSmart - Personal Expense Tracker

Course: ICS 2300 Mobile Applications Design and Development

Group: Group 2

1. Project Overview & Problem Statement

The **SpendSmart** application was developed to address the growing issue of financial illiteracy among young adults. Many individuals struggle with "invisible spending"—the accumulation of small, untracked daily expenses that lead to budget deficits.

Our goal was to create a tool that combines low-friction data entry with high-impact visual analytics to promote disciplined saving habits. The app provides immediate real-time awareness of financial health through an integrated dashboard and categorization engine.

Official Repository: https://github.com/Shemira01/MADD-Project-2.git

2. Technical Implementation & Architecture

To ensure scalability and performance, the group moved away from standard linear development to a **Modern Android Tech Stack**.

A. MVVM Design Pattern (Model-View-ViewModel)

The app utilizes the **MVVM pattern** to decouple business logic from the UI.

- **ViewModel (`ExpenseViewModel`):** Manages UI-related data in a lifecycle-conscious way, allowing data to survive configuration changes like screen rotations.
- **LiveData:** Used to provide real-time updates to the Dashboard. When a new expense is saved, the Dashboard observers trigger an automatic refresh without the user manually reloading the page.

B. Data Persistence (Room Database)

Unlike basic apps that lose data on exit, SpendSmart implements the **Room Persistence Library**:

- **Entity Mapping:** The `Expense` class acts as a database table.
- **DAO (Data Access Object):** A specialized interface (`ExpenseDao`) provides the SQL abstraction for CRUD (Create, Read, Update, Delete) operations.

- **Asynchronous Execution:** Database operations are handled on background threads to ensure the UI remains responsive and "jank-free."

C. Single Activity Architecture

The app uses a centralized `MainActivity` as a fragment host. Navigation is managed through a `BottomNavigationView` linked to a specialized `loadFragment` orchestration logic, significantly reducing memory overhead compared to a multi-activity approach.

3. Module Breakdown

Page 1: Dashboard (Intelligent Oversight)

- **Budget Tracking:** Displays the "Remaining Balance" by calculating the delta between the user's income and total expenses stored in Room.
- **Visual Status Indicator:** A dynamic `ProgressBar` that changes color (Green → Yellow → Red) based on percentage-of-budget consumed.
- **History Feed:** A `RecyclerView` utilizing a custom `TransactionAdapter` to list the 5 most recent transactions with high-speed layout inflation.

Page 2: Add Expense (Data Acquisition)

- **Material 3 UI:** Utilizes `TextInputLayout` with the `OutlinedBox` style and floating labels for a premium aesthetic.
- **UX Optimization:** Implemented specialized focus-management logic (`requestFocus`) that automatically resets the form and re-engages the soft keyboard after a save, facilitating rapid data logging.
- **Input Mapping:** Captures complex strings, dates, and spinner selections, parsing them into an immutable `Expense` object.

Page 3: Insights (Statistical Analysis)

- **Categorization Engine:** Groups expenditures by type (e.g., Food, Transport, Rent).
- **Relative Distribution:** Calculates category totals as a percentage of the grand total, displayed via horizontal consumption bars to help users identify high-leakage spending areas.

4. Team Roles & Task Allocation Matrix

Member Name	Project Role	Specific Technical Deliverable
Shemiramoth Mugo	**Technical Lead**	Architecture Backbone, Repository Management, and Form Reset Logic.
Lee Kariuki	**Lead UI Designer**	Material 3 Styling, XML Layouts, and Theme/Color Configuration.

Linet Mungai	**Input System Dev**	Entry Form Data Capture, Object Mapping, and Data Parsing logic.
Felix Wanyoike	**Validation Lead**	Field Error Handling (Regex), Validator Utility, and Currency Formatting.
Cecil Kioko	**Database Manager**	Room Persistence Setup, DAO implementation, and CRUD Operations.
Emmanuel Mwangangi	**Math & Logic Dev**	Spending Analysis Algorithms and Budget Percentage Calculations.
Gabriella Muthoni	**Visualization Dev**	Dashboard UI Binding, Progress Bar logic, and Insights Breakdown UI.
Samuel Kuria	**QA & Tech Writer**	Physical Device Testing, and Final Documentation.

5. Deployment & Performance Verification

The application was deployed on physical Android devices to verify stability:

1. **Responsiveness:** The `ConstraintLayout` XML structures were tested across different screen densities to ensure UI consistency.
2. **Data Integrity:** Verified that the SQLite database correctly persists data across app restarts.
3. **Efficiency:** The use of `BudgetMathUtils` ensures that complex summations do not block the main UI thread.

6. Conclusion

By integrating **Room Database**, **MVVM architecture**, and **Material Design 3**, Group 2 has delivered a robust, production-ready financial tool. **SpendSmart** effectively demonstrates the team's ability to implement professional-grade Android components into a unified, user-centric application.

Prepared by: Samuel Kuria